Software Requirements Engineering: Comprehensive Exam Master Guide & Paper Solution Manual

1. Executive Exam Orientation & Core Concepts

This study guide provides a complete, highly structured master framework for the **Software Requirements Engineering** examination. Grounded directly in the lecture series and mid-term examination standards, this document equips students to rapidly analyze, solve, and write full-mark answers for both conceptual and scenario-based exam questions.
Requirements Engineering (RE) is the systematic process of discovering, analyzing, documenting, and maintaining software requirements. As articulated by Fred Brooks in software engineering literature, deciding what to build is the hardest single part of building a software system. Misspecifying requirements introduces flaws that cripple the final software product and incur severe costs if rectified late in the development lifecycle.

2. Detailed Module Notes (RE&S Handouts 1–7)

Module 1: Foundations of Software Requirements & Functional Requirements

1. Definition and Distinction of Requirements

A requirement represents a complete specification of the desired external behavior of a software system without describing its internal implementation ("what" the system will do versus "how" it will do it).

- **Webster's Dictionary Distinction** : As stressed in the introductory lectures, a critical distinction must always be maintained between what is *wanted* by a stakeholder and what is genuinely *needed* .
- **IEEE Std 729 Definition** : A requirement is defined formally as a condition or capability that must be met or possessed by a system to satisfy a contract, standard, specification, or other formally imposed document.
- **Sources of Requirements** : Requirements originate from multiple sources including system stakeholders (anyone affected by the system), existing legacy systems, organizational documentation, and domain/business regulations.

2. Functional Requirements (FRs)

Functional requirements express the behavioral aspects and explicit services of the system. They detail how the system reacts to specific inputs, how it behaves in particular situations, sequencing, parallelism, and abnormal exception handling.

- **Key Characteristics** : Functional requirements must be complete and consistent. They form the primary focus of end-users and customers during project initiation.
- **Concrete Examples from Course Materials** :
- *Mathematical Calculation* : "The system shall solve a quadratic equation using the formula $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$." *(Note: As noted in class, this*

requirement suffers from incompleteness because it fails to specify system behavior when $a = 0$.)

- *Database Search* : "The user shall be able to search either the entire database of patients or select a subset from it (admitted patients, or patients with asthma, etc.)."
- *Document Viewer* : "The system shall provide appropriate viewers for the user to read documents in the document store." *(Note: Ambiguous because it does not define file formats or viewer specifications.)*
- *Unique Identification* : "Every order shall be allocated a unique identifier (ORDER_ID) which the user shall use to access that order."
- *Business Logic* : "The system shall allow customers to return non-perishable items within fifteen days of the purchase. A customer must present the original sale receipt to return an item."

Module 2: Non-Functional Requirements & McCall's Quality Model

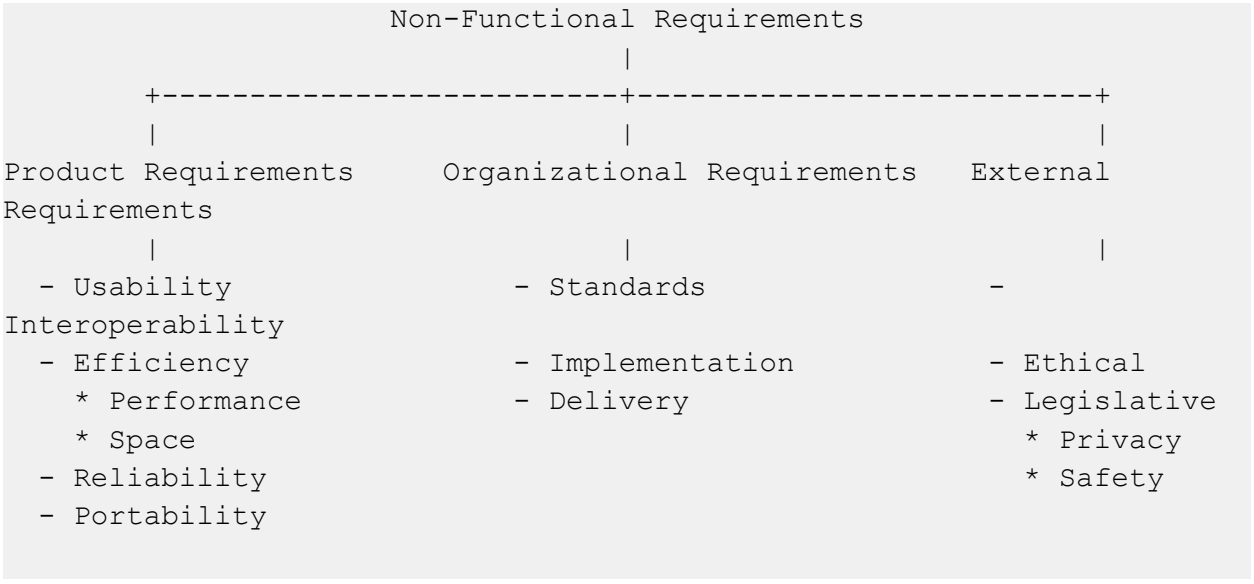
1. Nature of Non-Functional Requirements (NFRs)

Non-functional requirements specify constraints on the system as a whole rather than specific individual functions. They capture the *emergent behavior* of the system, such as performance, timing, reliability, security, maintainability, accuracy, and compliance with standards.

- **Criticality** : NFRs are frequently more critical than individual functional requirements. As documented in the course handouts, if an aircraft system fails its reliability requirements, it will not be certified as safe; if a real-time system fails performance bounds, its core control functions become unusable.
- **Origins** : NFRs arise from user needs, budget constraints, organizational policies, hardware/software interoperability requirements, and external safety or privacy legislation.

2. Taxonomy of Non-Functional Requirements

Non-functional requirements are categorized into three major branches:



Analytical Breakdown of NFR Categories:
NFR Category,Sub-Category,Description & Course Examples
Product Requirements,Usability,User interface accessibility and operational ease.
,Efficiency,"Comprises Performance (e.g., ""The system shall allow 100,000 hits per minute on the website"") and Space (memory/storage limits)."
,Reliability,"System uptime and failure tolerance (e.g., ""The system shall not have downtime exceeding 1 second for 1,000 hours of continuous execution"")."
,Portability,Ease of transferring the system across different hardware/OS platforms.
Organizational Requirements,Standards,"Compliance with engineering standards (e.g., ""Development deliverables shall conform to MIL-STD-2167A"")."
,Implementation,"Process constraints (e.g., ""Sub-contracted work shall comply with Capability Maturity Model CMM"")."
,Delivery,Timelines and delivery mechanisms for software drops.
External Requirements,Interoperability,Integration guidelines with third-party software/hardware systems.
,Ethical,Compliance with ethical norms and organizational integrity.
,Legislative,"Legal mandates, subdivided into Privacy (e.g., ""Library system shall not disclose member personal data except to system administrators"") and Safety ."

3. McCall's Quality Factors Model

McCall's model organizes software quality factors into three main operational perspectives:

1. **Product Operation** : Correctness, Reliability, Efficiency, Integrity, Usability.
2. **Product Revision** : Maintainability, Flexibility, Testability.
3. **Product Transition** : Portability, Reusability, Interoperability.

Module 3: NFR Metrics, Domain Requirements, Inverse Requirements & Constraints

1. Quantifying NFR Goals via Verifiable Metrics

NFRs stated as qualitative goals (e.g., "the system should be easy to use" or "rapid user response") are unverifiable and lead to misinterpretation. To be objectively tested, goals must be converted into quantitative metrics.
Quantitative Metrics Matrix for NFR Verification:
Quality Property,Quantitative Measure / Metric
Speed,1. Processed transactions per second 2. Response time 3. Screen refresh time
Size,1. Kilobytes (KB/MB) 2. Number of Function Points
Ease of Use,1. Training time required for users 2. Number of help frames
Reliability,1. Mean Time To Failure (MTTF) 2. Probability of unavailability 3. Rate of failure occurrence (ROCOF) 4. Availability percentage
Robustness,1. Time to restart after failure 2. Percentage of events causing failure 3. Probability of data corruption upon failure
Portability,1. Percentage of target-dependent statements 2. Number of target systems supported

- **Verifiable Conversion Example** :
- *Unverifiable Goal* : "The system should be easy to use by experienced controllers and organized to minimize user errors."

- *Verifiable NFR* : "Experienced controllers shall be able to use all system functions after a total of two hours' training. After this training, the average number of errors made by experienced users shall not exceed two per day."

2. Domain Requirements

Domain requirements derived directly from the application domain reflect fundamental operational characteristics of the business or scientific environment. They can be either functional or non-functional.

- **Challenges** : Domain experts often take domain knowledge for granted and fail to explicitly state these requirements. Absence of domain requirements can cause complete project failure.
- **Course Examples** :
- *Insulin Pump System* : "System safety shall be assured according to standard IEC 60601-1: Medical Electrical Equipment – Part 1: General Requirements for Basic Safety and Essential Performance." (Constrains both design and development process).
- *Automated Train Protection System* : Train deceleration is computed using domain physics equations: $$D_{\text{train}} = D_{\text{control}} + D_{\text{gradient}}$$ where $D_{\text{gradient}} = \frac{9.81\,\text{m/s}^2 \times \text{compensated gradient}}{\alpha}$, where constants are known for specific train classes.
- *Sales & Banking Domain Rules* : In commission sales, negative commission does not exist. In banking, standard accounts prohibit overdrawing while specialized accounts explicitly allow it.

3. Inverse Requirements

Inverse requirements specify explicitly what the system shall **not** do. They are used when stakeholders want to guard against undesirable system behavior.

- *Example* : "The system shall not use red color in the user interface whenever it is asking for inputs from the end-user."

4. Design and Implementation Constraints

Design constraints impose strict boundaries within which the software architecture and implementation must operate.

- *Examples* : "The system shall be developed using the Microsoft .NET platform" or "The system shall be developed using open-source tools and run on the Linux operating system."

Module 4: Views of Requirements & Natural Language Problems

1. User/Customer Requirements vs. System Contract Requirements

Requirements exist at different levels of detail depending on the audience:

- **User/Customer Requirements** : Stated in natural language, simple forms, or high-level diagrams. Accessible to non-technical users. Avoids internal design choices while providing the rationale for business needs.

- **System Contract Requirements** : Comprehensive, detailed technical descriptions that form the basis of the legal contract between client and developer. Used by system architects and software engineers as the baseline for system design.

2. Problems of Natural Language Specifications

Expressing requirements purely in natural language introduces five primary structural defects:

1. **Lack of Clarity (Ambiguity)** : Words or sentences open to multiple interpretations. (Course example: Escalator signs stating "Shoes must be worn" and "Dogs must be carried" lead to absurd literal interpretations).
2. **Requirement Confusion** : Mixing functional requirements with non-functional quality goals (e.g., ISO quality models listing functional suitability under non-functional attributes).
3. **Requirement Amalgamation** : Packaging several distinct requirements into a single unstructured sentence (e.g., specifying Bluetooth pairing, song download limits, and screen refresh times in one statement).
4. **Over-Flexibility / Multiple Interpretations** : The same concept expressed in completely different ways across a document, creating inconsistency.
5. **Lack of Modularity** : Natural language text cannot be easily modularized; assessing the impact of a single requirement change requires reviewing the entire specification document.

3. Ambiguity Case Study: Triplicated Message Voting

Consider the critical system requirement: *"The message must be triplicated. The three copies must be forwarded through three different physical channels. The receiver accepts the message on the basis of a two-out-of-three voting policy."*

1. **Ambiguities Identified in Course Analysis** :
2. Does the receiver accept the message immediately upon receiving 2 identical copies, or must it wait for the 3rd copy to arrive?
3. What action does the receiver take if all 3 incoming messages are completely different?

Module 5: Requirements Engineering Process & Activities

1. Inputs and Outputs of the RE Process

The RE process is a structured sequence that transforms stakeholder needs into formal specifications.

Inputs and Outputs Breakdown:

Process Phase,Item,Description
Inputs,Existing System Information,Information on legacy systems to be replaced or integrated.
,Stakeholder Needs,Operational requirements gathered from system users and managers.
,Organizational Standards,Quality management and engineering process guidelines.
,Regulations,"External health, safety, privacy, and legal frameworks."
,Domain Information,Technical knowledge regarding the application domain.
Outputs,Agreed Requirements,Stakeholder-approved statements of system behavior.
,System Specification,Detailed technical baseline for implementation.

,System Models,"Graphical analysis models (Data Flow Diagrams, Object Models, Process Diagrams)."

2. The Five Core RE Activities

1. **Requirements Elicitation** : Discovering and acquiring requirements through stakeholder consultation, document analysis, domain studies, and prototyping.
2. **Requirements Analysis and Negotiation** : Classifying requirements, resolving stakeholder conflicts, and prioritizing features based on technical feasibility and budget limits.
3. **Requirements Specification** : Formulating agreed requirements into tangible documents and models (Software Requirements Specification - SRS).
4. **Requirements Validation** : Reviewing specifications for consistency, completeness, and correctness to detect errors before development.
5. **Requirements Management** : Pervasive lifecycle activity covering traceability, version control, and change impact analysis.

3. Two Main Tasks of RE

- **Problem Analysis** : Understanding the real-world problem and user context.
- **Product Description** : Creating a complete specification of the desired external behavior of the software system.

Module 6: Roles, Actors & Role-Action Diagrams (RAD)

1. Roles and Stakeholders in RE

Actors in RE are defined by their functional roles rather than individual identities. Key stakeholders include:

- **Domain Expert** : Provides domain-specific knowledge and operational rules.
- **Requirements Engineer** : Elicits, analyzes, specifies, and validates system requirements.
- **End-User** : Interacts directly with the operational software post-delivery.
- **Software Engineer** : Translates specifications into software code and prototype models.
- **Project Manager** : Manages project scheduling, resource allocation, and budgeting.
- **Business Analyst** : Analyzes organizational business processes and aligns software capabilities.

2. Role-Action Diagram (RAD) for Software Prototyping

A Role-Action Diagram models the sequence of actions and maps the associated roles responsible for executing each action during prototyping.
Sequence of Prototyping Actions & Role Mapping:
$$\text{Understand Problem} \longrightarrow \text{Establish Outline Requirements} \longrightarrow \text{Select Prototyping System} \longrightarrow \text{Develop Prototype} \longrightarrow \text{Evaluate Prototype}$$
Role Responsibilities per Action:
Action Sequence,Participating Roles,Action Objective

1. Understand Problem,"Requirements Engineer, Domain Expert, End-User",Analyze application domain problems and user needs.
2. Establish Outline Requirements,"Requirements Engineer, End-User",Define initial high-level requirements.
3. Select Prototyping System,"Software Engineer, Project Manager","Choose prototyping tools, platform, and scope."
4. Develop Prototype,"Requirements Engineer, Software Engineer",Build executable prototype model.
5. Evaluate Prototype,"End-User, Domain Expert, Requirements Engineer, Software Engineer",Evaluate prototype against user expectations; feed back into requirements refinement.

Module 7: Social, Organizational & Cultural Issues in RE

1. Social & Organizational Dynamics

Requirements Engineering is fundamentally a social process involving three major groups: the **Client Organization** , the **Requirements Team** , and the **Development Team** .

- **Issues within Client Organization** : Competing internal divisions, political power structures, rank/status of stakeholders, and the *Say-Do problem* (the conflict between what users say they do and what they actually do due to tacit knowledge).
- **Issues between Client and Requirements Team** : Financial arrangements, confidentiality, legal obligations, and denial of information.

2. Cultural & Global Issues

Global software development introduces social and cultural frictions:

- **Terminology & Language Differences** :
- *Numerical Meaning* : In North America / scientific usage, a "billion" is 10^9 ($1,000,000,000$), whereas historically in European/international contexts it denoted 10^{12} ($1,000,000,000,000$).
- *Colloquial Expressions* : Sports lingo (e.g., "Fielding set hai") or regional terminology (e.g., "hockey" referring to field hockey in South Asia versus ice hockey in North America) causes communication misunderstandings.
- **Time Zone Differences** : Working hours separated by 8+ hours complicate live communication, tempting analysts to make unverified assumptions.
- **Religious, Racial, and Business Factors** : Insensitive language regarding ethnic/religious groups severely hampers collaboration. Business practices (e.g., Friday prayers, Ramadan hours, auction bidding conventions) must be respected.

3. Mid-Semester Exam Paper: Model Solutions & Breakdown

This section presents full-mark model answers for the 8 questions from the **IMSciences Mid-Semester Examination (Fall 2023)** for *Software Requirements Engineering / Specifications* (Instructor: Hamood ur Rehman Durrani).

Question 1 (3 Marks)

Prompt : *How would you define the term 'Requirements'? What are the problems posed by specifying Requirements in natural language?*

Model Answer (3/3 Marks):

1. Definition of Requirements : A requirement is a complete specification of a condition or capability that a software system must meet or possess to satisfy a contract, standard, specification, or formally imposed document (IEEE Std 729). It represents what the system shall do externally without dictating internal design choices.
2. Problems Posed by Natural Language Specifications : Specifying requirements using natural language leads to five major problems:

1. **Lack of Clarity (Ambiguity)** : Phrases are open to multiple interpretations (e.g., "Shoes must be worn").
2. **Requirement Confusion** : Mixing functional requirements with non-functional quality goals.
3. **Requirement Amalgamation** : Combining several distinct functional requirements into a single complex sentence.
4. **Over-Flexibility** : Expressing the same concept in completely different ways across the document.
5. **Lack of Modularity** : Difficulty in partitioning text, making change-impact analysis hard.

Question 2 (4 Marks)

Prompt : *The term 'System' has been used from varying standpoints. How would you describe the term 'System' from Requirements Engineering (RE) perspective keeping in view the four systems worlds discussed in the class?*

Model Answer (4/4 Marks):

1. RE Standpoint on 'System' : From a Requirements Engineering perspective, a system is not merely software code; it is a composite entity comprising software, hardware, operational processes, and business rules designed to solve a problem within an organizational environment.
2. Description across the Four System Worlds :

- **User World** : Represents the domain context, operational tasks, and real-world needs of end-users.
- **Developer / Programmer World** : Represents the technical implementation, code modules, architecture, and software constructs.
- **Real / Customer World** : Represents business objectives, economic value, organizational structures, and external constraints.
- **System World (Integration)** : From an RE perspective, the requirement specification acts as the bridge connecting these worlds. It translates needs from the user/customer real-world into actionable hardware, software, and business process constraints for the developer world.

Question 3 (4 Marks)

Prompt : *What are different types of Requirements? Which type of requirements are main cause of the inconsistency in requirements and why?*

Model Answer (4/4 Marks):

1. Five Types of Software Requirements :

1. **Functional Requirements** : Statements of services and functions the system must provide.
2. **Non-Functional Requirements** : System-wide constraints on performance, reliability, security, etc.
3. **Domain Requirements** : Fundamental requirements derived from the application domain.
4. **Inverse Requirements** : Statements specifying what the system shall *not* do.
5. **Design and Implementation Constraints** : Restrictive guidelines on platforms, tools, or languages.

2. Main Cause of Inconsistency : **Non-Functional Requirements (NFRs)** are the primary cause of inconsistency in requirements specifications.

Why NFRs Cause Inconsistency :

- **Qualitative Goal Framing** : NFRs are frequently expressed as general, qualitative goals (e.g., "fast response" or "user-friendly") rather than objectively verifiable metrics, making verification difficult.
- **Conflicting Stakeholder Priorities** : NFRs stem from diverse stakeholders with competing goals (e.g., security teams demanding complex authentication versus end-users demanding instant access; or high reliability versus strict budget limits).

Question 4 (4 Marks)

Prompt : *What are the three major categories of Non-Functional requirements? Name the types within each category.*

Model Answer (4/4 Marks):

The three major categories of Non-Functional Requirements and their sub-types are:

- **Product Requirements** : Specify product behavior constraints.
- *Usability Requirements*
- *Efficiency Requirements* (further subdivided into **Performance** and **Space** requirements)
- *Reliability Requirements*
- *Portability Requirements*
- **Organizational Requirements** : Derived from organizational policies and standards.
- *Standards Requirements*
- *Implementation Requirements*
- *Delivery Requirements*
- **External Requirements** : Derived from factors external to the organization.
- *Interoperability Requirements*
- *Ethical Requirements*

- *Legislative Requirements* (further subdivided into **Privacy** and **Safety** requirements)

Question 5 (4 Marks)

Prompt : *"RE processes vary radically from one organization to another, and even within an organization in different projects." What are the factors that contributes to the variability of the RE process?*

Model Answer (4/4 Marks):

Because there is no "ideal" Requirements Engineering process, RE processes are tailored according to the following key organizational and project factors:

1. **Technical Maturity** : The organizational maturity level (e.g., CMMI level) dictates the formal rigor, documentation, and process controls applied.
2. **Organizational Culture** : The established engineering practices, communication habits, and managerial culture of the organization.
3. **Disciplinary Involvement** : The specific engineering disciplines, methodologies, and standard tools mandated by the project domain.
4. **Application Domain** : The specific business or scientific field (e.g., banking vs. medical devices vs. aerospace).
5. **Software Project Type & Criticality** : Safety-critical systems (e.g., insulin pumps or flight control) require extensive validation and verification, whereas routine commercial applications require streamlined processes.

Question 6 (3 Marks)

Prompt : *What is meant by 'Sequencing' and 'parallelism' in requirements? How does it allow us to prioritize requirements?*

Model Answer (3/3 Marks):

1. Definitions :

- **Sequencing** : Refers to strict functional dependencies between requirements where the output of one requirement serves as the direct prerequisite or input for another requirement.
- **Parallelism** : Refers to independence between requirements, allowing them to be developed, tested, or executed concurrently without functional coupling.

2. Role in Requirement Prioritization :

- **Sequencing-based Prioritization** : Requirements with downstream dependencies must be assigned higher priority during elicitation and planning because dependent requirements cannot function without them.
- **Parallelism-based Prioritization** : Independent (parallel) requirements afford architectural flexibility. Prioritizing parallel streams minimizes module coupling and allows concurrent development across engineering teams.

Question 7 (4 Marks)

Prompt : *How does the Capability maturity model (CMM) for Requirements Engineering resolves the problems in the RE process?*

Model Answer (4/4 Marks):

1. Problems in RE Process : RE processes suffer from human, political, and social distortions—including stakeholder rank dominance, conflicting personal goals, miscommunication, lack of involvement, and informal unstandardized practices.
2. How CMM Resolves RE Problems : The Capability Maturity Model (CMM) introduces a standardized, structured framework operating across evolutionary maturity levels (Initial, Repeatable, Defined, Managed, Optimizing):

- **Standardized Practices** : Replaces informal habits with defined, repeatable procedures for requirements elicitation, documentation, and validation.
- **Objective Baseline** : Reduces individual stakeholder bias and political influence by enforcing objective metrics, formal reviews, and change control procedures.
- **Process Consistency** : Ensures organizational consistency across diverse projects, mitigating miscommunication and managing budget/schedule constraints.

Question 8 (4 Marks)

Prompt : *Draw a Role-Action Diagram for 'Software Prototyping' used from Requirements Elicitation perspective. Give a brief description of the key roles.*

Model Answer (4/4 Marks):

1. Role-Action Flow Diagram :

```
[Actions]:   Understand Problem ---> Establish Outline Req. --->
Select Prototyping System ---> Develop Prototype ---> Evaluate
Prototype
                     |                              |
|                        |                      |
[Roles]:     - Req. Engineer              - Req. Engineer             -
SW Engineer           - Req. Engineer        - End-User
             - Domain Expert              - End-User                  -
Project Manager       - SW Engineer          - Domain Expert
             - End-User
- Req. Engineer

- SW Engineer
```

2. Description of Key Roles :

- **Domain Expert** : Provides domain knowledge and validates operational business rules.
- **Requirements Engineer** : Elicits, analyzes, documents, and coordinates overall requirements.
- **End-User** : Evaluates prototype iterations to confirm usable requirements.
- **Software Engineer** : Designs and constructs the executable prototype model.
- **Project Manager** : Plans resources, schedules, and oversees the prototyping process.
- **Business Analyst** : Maps organizational business processes into software requirements.

4. High-Yield Short-Answer Quiz & Answer Key

Practice Quiz Questions

1. What is the main difference between user requirements and system contract requirements?
2. Why is the requirement "The system shall solve a quadratic equation using $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ " considered incomplete?
3. What are the three operational dimensions of McCall's Quality Factors Model?
4. Convert the qualitative goal "The system should recover quickly from failure" into a verifiable NFR.
5. Provide an example of a domain requirement in medical software.
6. What is an inverse requirement? Give one example.
7. Identify the ambiguity in the escalation requirement "Dogs must be carried."
8. Name the five core activities of the Requirements Engineering process.
9. What is the *Say-Do problem* in Requirements Engineering?
10. How do numerical definitions of a "billion" differ internationally and why does this matter in global RE?

Answer Key with Explanations

1. **User vs. System Contract Requirements** : User requirements are high-level statements in natural language for non-technical clients; System contract requirements are comprehensive technical specifications forming a legal contract for developers.
2. **Incompleteness in Quadratic Formula Requirement** : It fails to specify system behavior or exception handling when the coefficient $a = 0$ (which leads to division by zero).
3. **McCall's Three Dimensions** : Product Operation, Product Revision, and Product Transition.
4. **Verifiable NFR Conversion** : "In the event of a system crash, the system shall recover full operational state within 30 seconds, with zero probability of database corruption."
5. **Medical Domain Requirement** : Compliance with standard IEC 60601-1 for medical electrical equipment safety in insulin pump design.
6. **Inverse Requirement** : Defines what the system shall *not* do (e.g., "The system shall not store unencrypted passwords").
7. **Escalator Ambiguity** : Literally implies that every passenger must carry a dog to use the escalator.
8. **Five RE Activities** : Elicitation, Analysis & Negotiation, Specification, Validation, Management.
9. **Say-Do Problem** : The disconnect between what users verbally state they do versus their actual work practices due to implicit/tacit knowledge.
10. **Billion Terminology Conflict** : North America/science defines a billion as 10^9 , whereas traditional European usage defined it as 10^{12} , creating software calculation errors if misaligned.

5. Comprehensive Essay & Synthesis Prompts

(Use these prompts for self-testing high-level synthesis concepts before the exam.)

1. **Comparative Analysis of Requirements Types** : Compare Functional, Non-Functional, Domain, and Inverse Requirements. Explain with examples how a domain requirement can simultaneously act as a non-functional constraint.
2. **Natural Language Vulnerabilities in SRS** : Discuss why natural language remains the predominant format for requirements specifications despite its inherent defects. Evaluate formal modeling alternatives (e.g., UML).
3. **Role of Human and Organizational Dynamics** : Analyze how political influence, stakeholder status, and organizational culture distort the RE process. Detail how process models like CMM mitigate these friction points.
4. **NFR Metric Engineering** : Explain the engineering challenges associated with quantifying non-functional goals. Discuss the cost-benefit trade-off of quantitative verification.
5. **Global & Cultural Requirements Engineering** : Evaluate how geographical dispersion, time zones, and language dialects affect requirements elicitation and negotiation in cross-border software projects.

6. Glossary of Core Requirements Engineering Terminology

- **Requirements Engineering (RE)** : The systematic process of establishing the services required of a system and the constraints under which it operates and is developed.
- **IEEE Std 729** : IEEE standard defining a requirement as a condition or capability needed by a user to solve a problem or achieve an objective.
- **Functional Requirement** : Statement of a specific service or behavior the system must provide.
- **Non-Functional Requirement** : Constraint on system properties or emergent behaviors (performance, safety, reliability).
- **Domain Requirement** : Requirement derived directly from the application domain's operational laws or physics.
- **Inverse Requirement** : Explicit specification detailing behavior or actions the system must *not* execute.
- **Emergent Behavior** : System-level properties that emerge from the interaction of system components as a whole rather than from individual parts.
- **Role-Action Diagram (RAD)** : Process model visually depicting roles alongside their corresponding sequence of activities.
- **Say-Do Problem** : Difficulty stakeholders face in describing tasks they perform automatically due to tacit knowledge.
- **Requirement Traceability** : The capability to track a requirement from its origin through analysis, design, code, and test cases.
- **CMM (Capability Maturity Model)** : A process capability framework helping organizations formalize and optimize engineering processes.